

Prüfung Web-Grundlagen

Frage 1:

Ein Entwickler hat folgenden Code geschrieben, um Produkte von einer API zu laden:

```
fetch('https://shop.example.com/api/products')
  .then((response) => response.json())
  .then((data) => {
    document.getElementById('list').innerHTML = data.map(/* ... */);
  })
  .catch((error) => {
    console.error('Fehler beim Laden:', error);
  });
```

In der Konsole erscheint folgende Ausgabe:

```
GET https://shop.example.com/api/products 504 (Gateway Timeout)
Fehler beim Laden: SyntaxError: Unexpected token '<' at position 0
```

Die API liefert normalerweise JSON-Daten. Warum erscheint dieser `SyntaxError`? Was ist tatsächlich passiert, und wie könnte man den Code verbessern, um solche Fehler frühzeitig zu erkennen?

`response.json` versucht eine HTML Fehlerseite (beginnend mit "<") als JSON zu interpretieren. Um das zu vermeiden soll vor dem Parsen geprüft werden, ob die Response "OK" war.

Frage 2:

Was ist DNS und welche Rolle spielt es bei der Kommunikation im Web?

Domain Name System - sowie das Telefonbuch des Internets. Jedes mal, wo wir eine Seite abrufen (z.B. google.com) wird diese Domain Name in eine IP Adresse umgewandelt (und umgekehrt). Somit müssen sich Benutzer keine lange, schwer-merkbare Adressen merken.

Beim Aufruf einer Website löst DNS zuerst den Namen der Domain auf, bevor eine Verbindung zum Webserver hergestellt wird.

Frage 3:

Was ist der Unterschied zwischen Block-Level- und Inline-Elementen in HTML? Nennen Sie jeweils mindestens zwei Beispiele. Welche Verschachtelungsregel gilt zwischen Block- und Inline-Elementen?

Block Level Elemente erzeugen einen neuen Absatz und nehmen eine ganze Zeile platz.

Beispiele: div, p

Inline-Elemente bleiben im Textfluss und nehmen nur so viel Platz wie sie tatsächlich brauchen

Beispiele: span, a

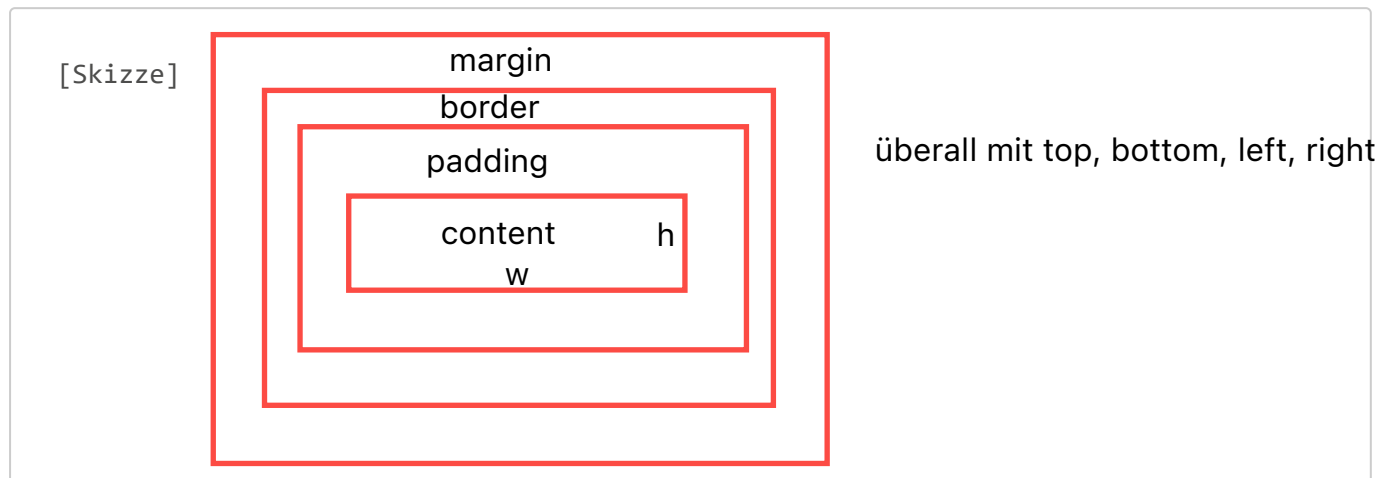
Frage 4:

Benennen und erklären Sie die Komponenten der folgenden URL:

<https://www.shop.com/pages/kontakt.html?lang=de#top>

Frage 5:

Was versteht man unter dem CSS Box-Modell? Zeichnen Sie eine Skizze des Box-Modells und beschriften Sie die vier Bestandteile: border, content, margin, padding.



Frage 6:

Welche Farbe haben die Paragraphen im folgenden HTML-Dokument?

```
<head>
  <style>
    .container p {
      color: red;
    }
    #wrapper p {
      color: green;
    }
    p.urgent {
      color: black;
    }
    #wrapper .container p.urgent {
      color: cyan;
    }
  </style>
</head>
<body>
  <div id="wrapper">
    <div class="container">
      <p>Paragraph 1</p>
      <p class="urgent">Paragraph 2</p>
      <p class="urgent" style="color: magenta;">Paragraph 3</p>
      <p style="color: yellow;">Paragraph 4</p>
    </div>
  </div>
</body>
```

- Paragraph 1: green
 - Paragraph 2: cyan
 - Paragraph 3: magenta
 - Paragraph 4: yellow
-

Frage 7:

Semantische HTML5-Elemente wie `<header>`, `<nav>` oder `<article>` sind wichtige Bestandteile moderner Webentwicklung.

Nennen Sie zwei konkrete Vorteile, die `<div>`-Elemente gegenüber semantischen Elementen haben können, und in welchen Situationen ein `<div>` die bessere Wahl ist.

1. Flexibilität für Layout / Styling

Divs sind gut dafür, Elemente zu gruppieren, wo die Gruppierung rein für Styling-Gründen benötigt wird.

2. Vermeidung falscher Semantik

Wenn es keine passende semantische Elemente gibt, dann soll man bevor man was unpassendes nehmen, eher eine div nehmen

Frage 8:

In der Browser-Konsole erscheint folgende Ausgabe beim Absenden einer Bestellung:

```
POST https://shop.example.com/api/orders 401 (Unauthorized)
Uncaught (in promise) Error: Request failed with status code 401
    at handleResponse (api.js:24)
    at async submitOrder (checkout.js:156)
```

Handelt es sich um einen Client- oder einen Server-Fehler? Was ist die wahrscheinliche Ursache und wie würden Sie das Problem beheben?

Frage 9:

In der Browser-Konsole erscheint folgende Ausgabe beim Laden einer Produktliste:

```
GET https://shop.example.com/api/products?category=electronics 500 (Internal
Server Error)
Uncaught (in promise) Error: Request failed with status code 500
    at handleResponse (api.js:24)
    at async loadProducts (catalog.js:42)
```

Handelt es sich um einen Client- oder einen Server-Fehler? Liegt das Problem im eigenen JavaScript-Code? Was sollten Sie als nächstes tun?

Ein 500 Internal Server Error ist ein Server-Fehler. Das Problem liegt eigentlich nicht im Code, sondern im Backend oder der API.

Als nächstes sollte man den Request im Network-Tab prüfen, die Server-Logs untersuchen und die API unabhängig testen, um die Ursache zu finden.

Frage 10:

Folgende HTML-Seite zeigt im Browser eine komplett leere weiße Seite an:

```
<!DOCTYPE html>
<html lang="de">
<head>
  <meta charset="UTF-8">
  <title>Meine Seite</title>
  <script>
    console.log("Initialisierung...");
  </head>
<body>
  <h1>Willkommen auf meiner Webseite</h1>
  <p>Dieser Text sollte sichtbar sein.</p>
</body>
</html>
```

Finden und erklären Sie den Fehler. Warum wird der gesamte Inhalt nicht gerendert? Wie würden Sie ihn beheben?

Hier wird das Script Tag nie zugemacht und deshalb wird nichts gerendert (Browser interpretiert alles dannach noch als JS, statt HTML). Um den Fehler zu beheben müsste man also nur nach den console.log "</script>" schreiben